

Binarizzazione di una matrice

Progetto - Sistemi Concorrenti e Paralleli M

A.A. 2025-2026

Davide Chirichella - 0001222371

Contents



1	Proposta di progetto	4
1.a	Obiettivi	5
2	Implementazione Seriale	6
2.a	Algoritmo	7
3	Implementazione Parallela: MPI	10
3.a	Distribuzione del carico di lavoro	11
3.b	Allocazione e scambio delle Ghost Rows	13
3.c	Calcolo elementi dell'intorno e sogliatura	18
3.d	Raccolta risultati	20
4	Implementazione Parallela: OpenMP	21
4.a	Considerazioni generali	22
4.b	Scelte implementative	23
4.c	Algoritmo	25
5	Benchmark	26
5.a	Setup sperimentale	27
5.b	MPI	29

Contents (ii)



5.b.a Strong Scaling inter-nodo	29
5.b.b Weak Scaling inter-nodo	31
5.b.c Strong Scaling intra-nodo	33
5.b.d Weak Scaling intra-nodo	35
5.c OpenMP	37
5.c.a Strong Scaling	37
5.c.b Weak Scaling	39
6 Conclusioni	41
6.a Risultati ottenuti	42
6.a.a OpenMP (Shared Memory)	42
6.a.b MPI (Distributed Memory)	42

1 Proposta di progetto

Obiettivi



Trasformare una matrice **quadrata** in una matrice **binaria** basata sulla media locale di ogni elemento.

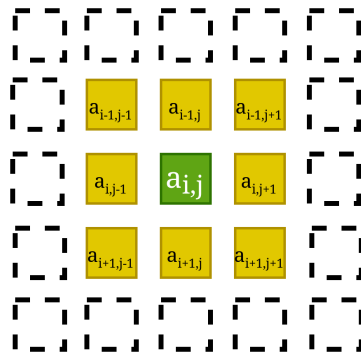
- **Matrice di Input (A)** di dimensione $N \times N$, con $N \geq 2000$.
- **Matrice di Output (T)** di dimensione $N \times N$ con valori $\{0, 1\}$.

1. **Media dell'intorno ($m_{i,j}$):** Media aritmetica degli elementi appartenenti all'intorno $I_{i,j}$ con la formula:

$$m_{i,j} = \frac{1}{9} \sum_{x=i-1}^{i+1} \sum_{y=j-1}^{j+1} a_{x,y}$$

2. **Sogliatura binaria:**

- Se $a_{i,j} > m_{i,j}$ allora $t_{i,j} = 1$
- Se $a_{i,j} \leq m_{i,j}$ allora $t_{i,j} = 0$



2 Implementazione Seriale

Algoritmo



Si allocano le matrici secondo la **Memorizzazione Row-Major**. L'elemento in posizione (i, j) può essere acceduto linearmente come: `A[i * N + j]`, e viene allocata nel seguente modo:

```
int *A = malloc(N * N * sizeof(int));  
int *T = malloc(N * N * sizeof(int));
```

- Si cicla lungo le righe e le colonne, definendo la somma dei valori `sum` e il numero di elementi `count`:

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        sum = 0;  
        count = 0;
```

Algoritmo (ii)



E si definiscono i confini dell'intorno, stabilendo “fino a quanto” si può uscire dalla cella corrente:

```
zmin = (i > 0) ? i - 1 : i;  
zmax = (i < N - 1) ? i + 1 : i;  
wmin = (j > 0) ? j - 1 : j;  
wmax = (j < N - 1) ? j + 1 : j;
```

Si cicla successivamente tra i valori dell'intorno:

```
for ( z = zmin; z <= zmax; z++) {  
    for ( w = wmin; w <= wmax; w++) {  
        sum += A[z * N + w];  
        count++;  
    }  
}
```


Algoritmo (iii)



Infine, si calcola la soglia binaria:

```
T[i * N + j] = (A[i * N + j] * count > sum) ? 1 : 0
```

Nota: È presente una leggera **ottimizzazione**, ovvero l'uso della formula $(A[i][j] * \text{count} > \text{sum})$ al posto della divisione $(\text{sum}/\text{count})$.

3 Implementazione Parallela: MPI

Distribuzione del carico di lavoro



Considerazioni:

- MPI adotta un modello a **scambio di messaggi**: necessità di comunicazione tra processi per elementi dell'intorno.

Calcolo dimensioni delle sotto-matrici

Poiché non è garantito che la dimensione della matrice N sia un multiplo esatto del numero di processi:

base_rows: Ogni processo riceve almeno $N / \text{num_proc}$ righe.

extra_rows: Le righe rimanenti $(N \% \text{num_proc})$ vengono ridistribuite.

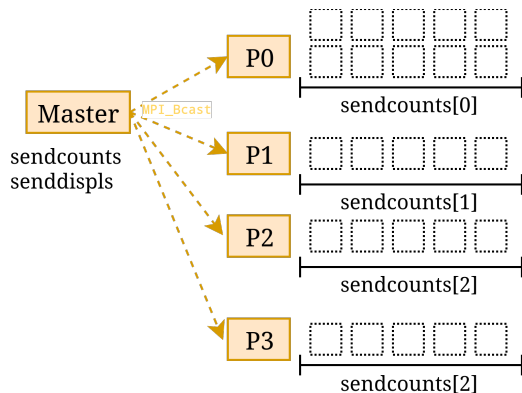
Distribuzione del carico di lavoro (ii)



Calcolo elementi di ogni processo, inviati in Broadcast:

sendcounts: Memorizza il numero totale di elementi assegnati a ogni processo.

senddispls: L'offset iniziale di ciascun processo.



Allocazione e scambio delle Ghost Rows

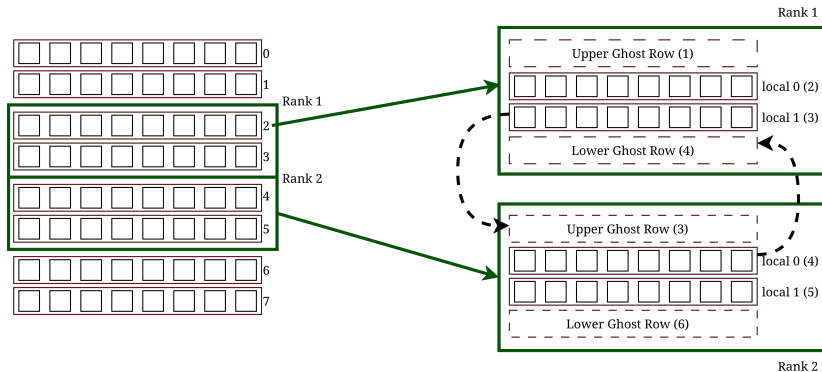


Ogni processo alloca quindi la propria porzione di righe:

```
my_rows = sendcounts[my_rank] / N
```

Dovrà tuttavia allocare due righe extra:

- **Upper Ghost Row:** Ospita l'ultima riga del processo precedente (P_{k-1}), se esiste.
- **Lower Ghost Row:** Ospita la prima riga del processo successivo (P_{k+1}), se esiste.



Allocazione e scambio delle Ghost Rows (ii)



Successivamente, ogni processo:

1. Determina chi sono processi adiacenti:

```
int up    = (my_rank > 0)           ? my_rank - 1 : MPI_PROC_NULL;  
int down  = (my_rank < num_proc - 1) ? my_rank + 1 : MPI_PROC_NULL;
```

2. Alloca la struttura dati per propria porzione di matrice con due righe extra:

```
int total_rows = my_rows + 2;  
int *my_A_plus_ghosts = calloc(total_rows * N, sizeof(int));
```

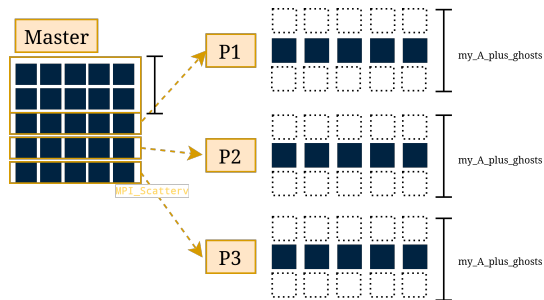
3. Definisce un puntatore per ogni sezione della memoria allocata:

```
int *upper_ghost = my_A_plus_ghosts;           // row -1  
int *local_data  = my_A_plus_ghosts + N;       // rows [0 ... my_rows-1]  
int *lower_ghost = my_A_plus_ghosts + (my_rows + 1) * N; // row + my_rows
```

Allocazione e scambio delle Ghost Rows (iii)



4. Il Master distribuisce le porzioni della matrice ai vari processi tramite una `MPI_Scatterv`:



```
MPI_Scatterv(  
    A, sendcounts, senddispls, MPI_INT,  
    local_data, sendcounts[my_rank], MPI_INT,  
    0, MPI_COMM_WORLD  
);
```

Approcci possibili per la comunicazione delle righe:

- `MPI_Recv` (bloccante) + `MPI_Ssend`: implementano una comunicazione sincrona di tipo **rendez-vous**.
- `MPI_Irecv` + `MPI_Isend`: permettono comunicazioni **asincrone**, richiedono `MPI_Wait`.
- `MPI_Sendrecv`: esegue simultaneamente invio e ricezione, ottimizzando la sincronizzazione.

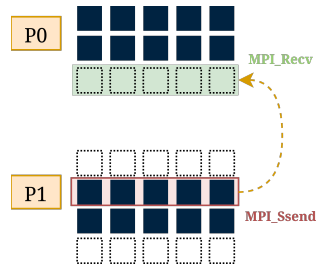
Allocazione e scambio delle Ghost Rows (iv)



Di seguito è mostrata l'implementazione mediante `Ssend` e `Recv`:

1. Ogni processo (tranne il primo) invia la propria riga di bordo superiore al vicino `up`:

```
if (my_rank > 0 && my_rows > 0) {  
    MPI_Ssend(local_data, N, MPI_INT, up, UP_TAG,  
MPI_COMM_WORLD);  
    MPI_Recv(upper_ghost, N, MPI_INT, up, DOWN_TAG,  
MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
}
```



2. Successivamente, si calcola l'indice di memoria dove inizia l'ultima riga locale per l'invio al processo sottostante:

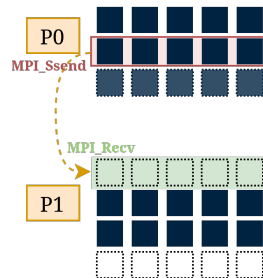
```
int send_down_offset = (my_rows > 0 ? (my_rows - 1) * N : 0);
```


Allocazione e scambio delle Ghost Rows (v)



3. L'invio tramite `Ssend` sincronizza i processi: una volta completata la trasmissione della riga di confine, il processo viene “sbloccato” per la ricezione dal nodo superiore della Ghost Row necessaria:

```
if (my_rank < num_proc - 1 && my_rows > 0) {  
    if (sendcounts[down] > 0) {  
        MPI_Recv(lower_ghost, N, MPI_INT, down, UP_TAG,  
MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
        MPI_Ssend(local_data + send_down_offset, N, MPI_INT,  
down, DOWN_TAG, MPI_COMM_WORLD);  
    }  
}
```



Calcolo elementi dell'intorno e sogliatura



Dopo lo scambio dei dati, ogni processo itera sulla propria porzione di matrice. Si ricorda che:

Indice (riga logica)	Contenuto
riga 0	Ghost row superiore
righe 1 ... my_rows	Righe di dati locali
riga my_rows + 1	Ghost row inferiore

Per compensare la presenza delle Ghost Rows, l'indice locale deve quindi essere traslato verticalmente:

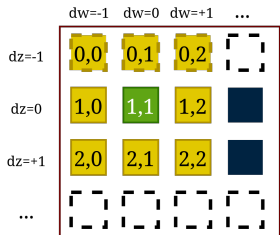
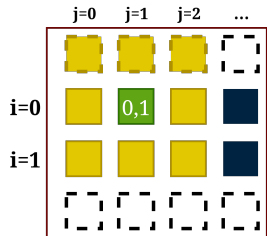
```
int zmin = (i == 0 && my_rank == 0) ? 0 : -1;           // limite superiore
int zmax = (i == my_rows - 1 && my_rank == num_proc - 1) ? 0 : 1; // limite inferiore
```

E orizzontalmente:

```
int wmin = (j > 0) ? -1 : 0;    // limite destro
int wmax = (j < N - 1) ? 1 : 0; // limite sinistro
```

Calcolo elementi dell'intorno e sogliatura (ii)

```
for (int dz = zmin; dz <= zmax; dz++) {  
    int rowIndex = i + 1 + dz;  
  
    for (int dw = wmin; dw <= wmax; dw++) {  
        int colIndex = j + dw;  
  
        sum += my_A_plus_ghosts[rowIndex * N +  
            colIndex];  
  
        count++;  
    }  
}  
  
my_T[i * N + j] = (local_data[i * N + j] * count > sum)?  
1 : 0;
```



Raccolta risultati



Si libera la memoria allocata per la matrice locale con le Ghost Rows:

```
free(my_A_plus_ghosts);
```

Il nodo Master invoca la primitiva `MPI_Gatherv` per collezionare i risultati parziali:

```
MPI_Gatherv(  
    my_T, (my_rows * N), MPI_INT,  
    T_raw, sendcounts, senddispls, MPI_INT,  
    0, MPI_COMM_WORLD  
);
```

4 Implementazione Parallela: OpenMP

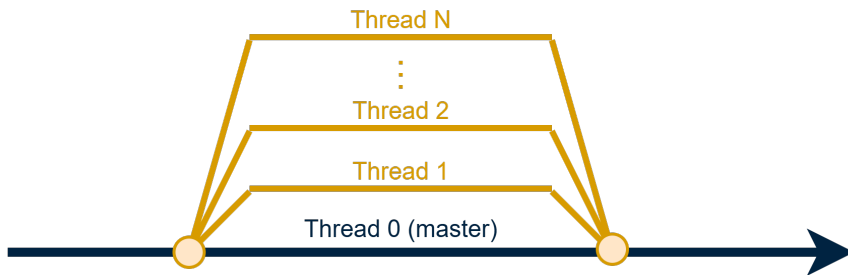
Considerazioni generali



L'implementazione OpenMP è significativamente più semplice a causa della sua gestione dei dati:

- Approccio **Shared Memory**, ogni thread può leggere direttamente l'intorno di un punto.
- Il compilatore si occupa di parallelizzare il codice mediante clausole singole `#pragma`

Poichè OpenMP segue il modello *Cobegin-Coend*, il Master thread dovrà creare dei worker thread al momento della direttiva `parallel`.



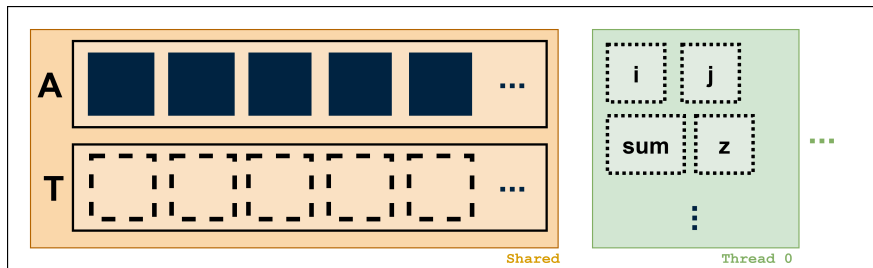
Scelte implementative



Le scelte adottate per l'implementazione dell'algoritmo parallelo hanno riguardato:

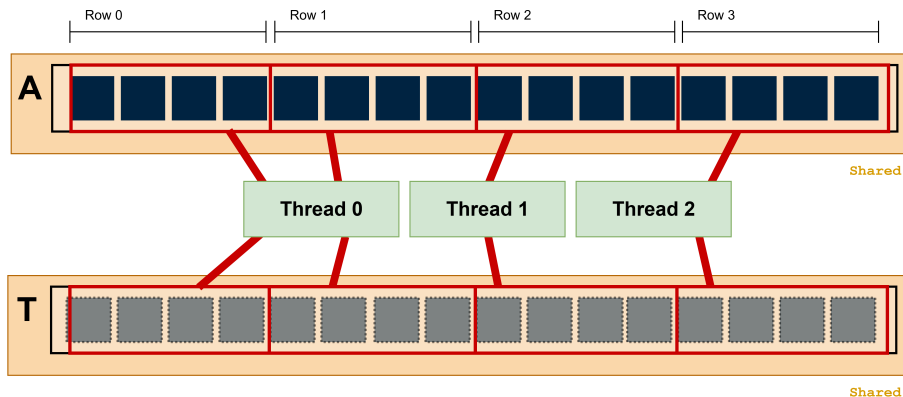
- **Visibilità delle Variabili:**

- ▶ Matrici A e T : allocate fuori dal ciclo e dichiarate `shared` tra tutti i thread.
- ▶ Variabili di supporto al calcolo locale (i , j , sum , $count$, etc...): dichiarate `private` (andranno successivamente inizializzate dai singoli thread).



- **Clausola `for`:** Permette di distribuire le iterazioni di un ciclo tra i thread, e si è scelto di farlo solo per il ciclo “esterno” (che itera sulle righe).

- Numero di thread: 3



Algoritmo



Si definiscono le clausole OpenMP che permettono di stabilire:

- L'assegnazione statica delle righe nel ciclo **for**;
- Numero di **thread** per processo (come parametro del programma);
- Variabili **condivise**;
- Variabile **private** (non inizializzate).

```
#pragma omp parallel for num_threads(num_threads) schedule(static) \  
shared(A, T, N) private(i, j, sum, count, zmin, zmax, wmin, wmax, z, w)
```

L'implementazione dell'algoritmo è la medesima della versione sequenziale:

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        sum = 0;  
        count = 0;  
        // ...
```

5 Benchmark

Setup sperimentale



Obiettivo

Valutare la scalabilità dell'algoritmo al variare delle risorse di calcolo e della dimensione del problema.

Analisi Effettuate

1. **Strong Scaling:** Dimensione del problema fissa ($N = 10.000$), unità di calcolo parallelo crescente ($P = 1...48$).
2. **Weak Scaling:** Dimensione del problema ($N = 5000\sqrt{P}$) e unità di calcolo parallelo ($P = 1...48$) crescenti.

Configurazioni testate:

1. **MPI intra-nodo** (più processi MPI indipendenti su un singolo nodo)
2. **MPI inter-nodo** (un processo MPI per nodo, con aumento progressivo del numero di nodi utilizzati)
3. **OpenMP** inter-nodo.

Setup sperimentale (ii)



Le direttive **Slurm** utilizzate sulla macchina *Galileo-100* del *Cineca* sono le seguenti:

Configurazione	Nodes	Tasks	CPU per Task	N
MPI , Strong scaling inter-nodo	1	1...48	1	10.000
MPI , Weak scaling inter-nodo	1	1...48	1	$5000\sqrt{P}$
MPI , Strong scaling intra-nodo	1...24	1 per nodo	1	10.000
MPI , Weak scaling intra-nodo	1...24	1 per nodo	1	$5000\sqrt{P}$
OpenMP , Strong scaling	1	1	48 (1...48 thread)	10.000
OpenMP , Weak scaling	1	1	48 (1...48 thread)	$5000\sqrt{P}$

5.b.a Strong Scaling inter-nodo

- All'aumentare di P , il numero di righe elaborate da ogni processo diminuisce, aumentando l'**overhead**.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	10000	3.7963	0.0048	1	1
2	10000	2.0473	0.0043	1.8544	0.9272
4	10000	1.1888	0.0054	3.1934	0.7983
8	10000	0.7644	0.0043	4.9664	0.6208
12	10000	0.6133	0.0014	6.1905	0.5159
16	10000	0.5108	0.0011	7.4318	0.4645
20	10000	0.4684	0.0025	8.105	0.4052
24	10000	0.4414	0.001	8.6	0.3583
48	10000	0.4002	0.0143	9.4866	0.1976

MPI (ii)

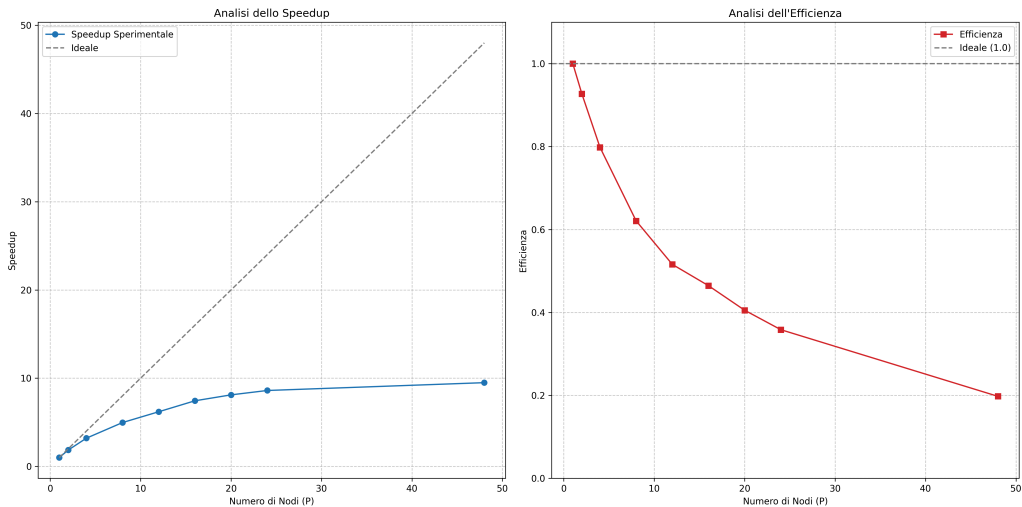


Figure 1: Strong Scaling inter-nodo - MPI

MPI (iii)



5.b.b Weak Scaling inter-nodo

- Nel caso di **weak scaling** si osserva un comportamento analogo.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	5000	0.9482	0.0016	1	1
2	7071	1.0245	0.0016	1.851	0.9255
4	10000	1.1913	0.0008	3.1836	0.7959
8	14142	1.5363	0.0017	4.9373	0.6172
12	17320	1.871	0.0041	6.0813	0.5068
16	20000	1.9983	0.0054	7.5919	0.4745
20	22360	2.2752	0.0099	8.3348	0.4167
24	24494	2.564	0.0068	8.8752	0.3698
48	34641	4.6097	0.0085	9.8732	0.2057

MPI (iv)

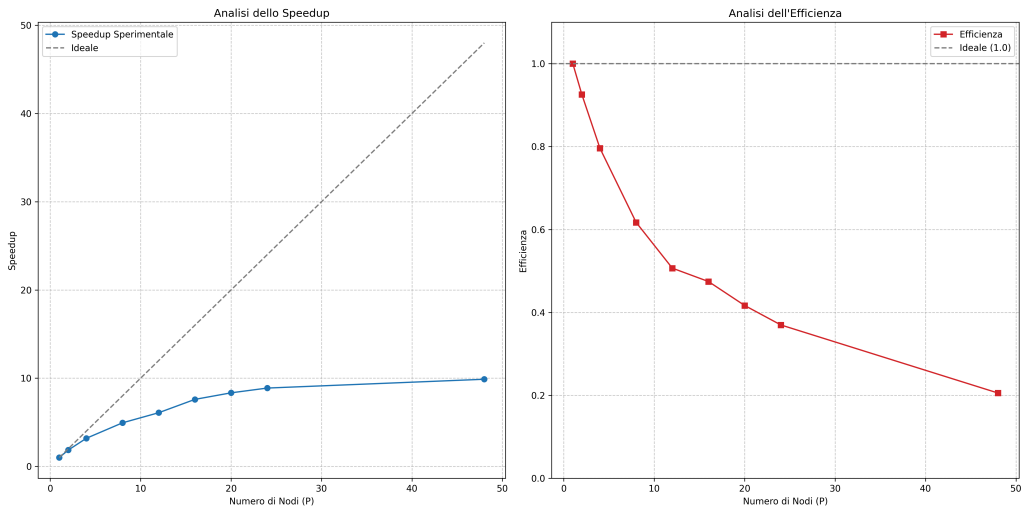


Figure 2: Weak Scaling inter-nodo - MPI

5.b.c Strong Scaling intra-nodo

- La similitudine tra il comportamento MPI **intra-nodo** e **inter-nodo** conferma il predominio dell'overhead di comunicazione rispetto al lavoro computazionale, che limita lo scaling.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	10000	3.6967	0.2585	1	1
2	10000	2.032	0.0207	1.8192	0.9096
4	10000	1.1396	0.0236	3.2438	0.811
8	10000	0.6721	0.1351	5.5006	0.6876
12	10000	0.5984	0.0011	6.1781	0.5148
16	10000	0.4511	0.0432	8.1946	0.5122
20	10000	0.4605	0.0543	8.0272	0.4014
24	10000	0.3882	0.0135	9.5235	0.3968

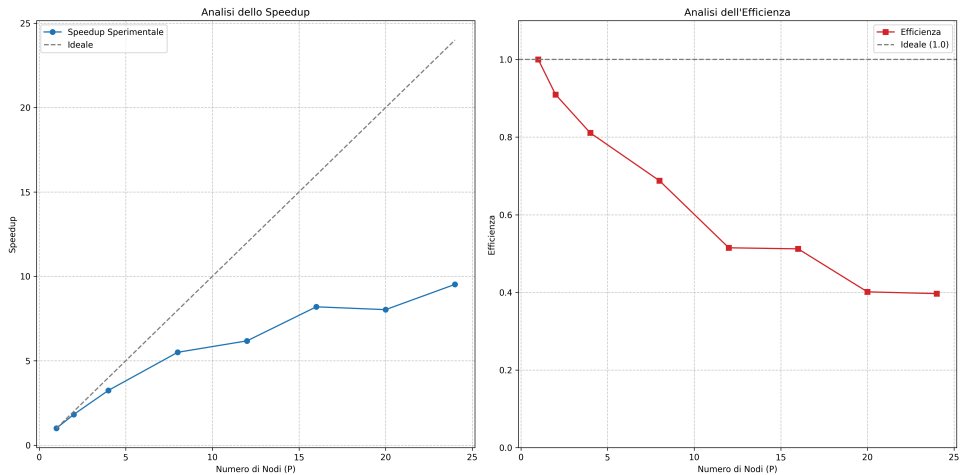


Figure 3: Strong Scaling intra-nodo - MPI

MPI (vii)



5.b.d Weak Scaling intra-nodo

- Si osserva anche qui un comportamento analogo al caso precedente, con benefici di scaling limitati.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	5000	0.9623	0.0235	1	1
2	7071	1.0243	0.0131	1.879	0.9395
4	10000	1.1515	0.0093	3.3429	0.8357
8	14142	1.4222	0.0342	5.4132	0.6767
16	20000	1.949	0.0812	7.9001	0.4938
20	22360	2.7806	0.0055	6.9219	0.3461
24	24494	2.7406	0.0685	8.4274	0.3511

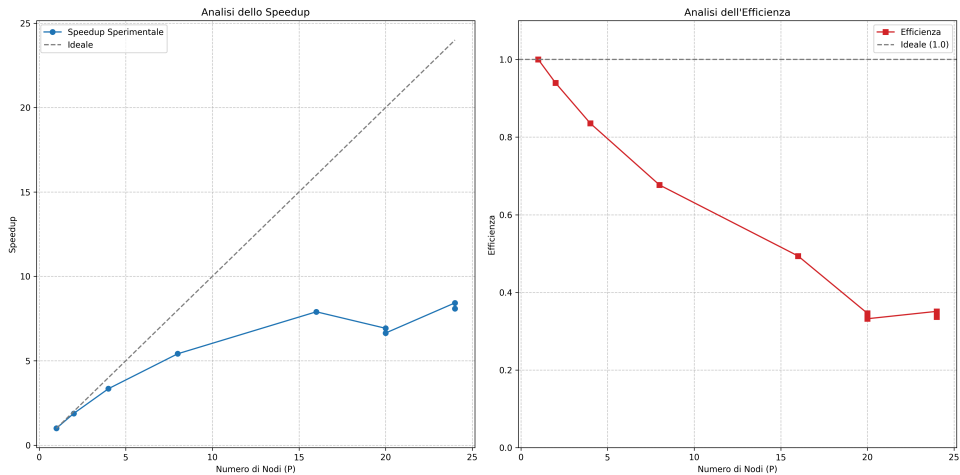


Figure 4: Weak Scaling intra-nodo - MPI

5.c.a Strong Scaling

- OpenMP mantiene un'efficienza estremamente elevata, sempre superiore al **90%**.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	10000	3.4676	0.0027	1	1
2	10000	1.7366	0.003	1.9967	0.9984
4	10000	0.8709	0.0036	3.9817	0.9954
8	10000	0.4379	0.0044	7.9195	0.9899
12	10000	0.2918	0.0016	11.8844	0.9904
16	10000	0.2194	0.0014	15.8064	0.9879
20	10000	0.177	0.0006	19.5958	0.9798
24	10000	0.1573	0.0025	22.0464	0.9186
48	10000	0.0801	0.0008	43.2907	0.9019

OpenMP (ii)

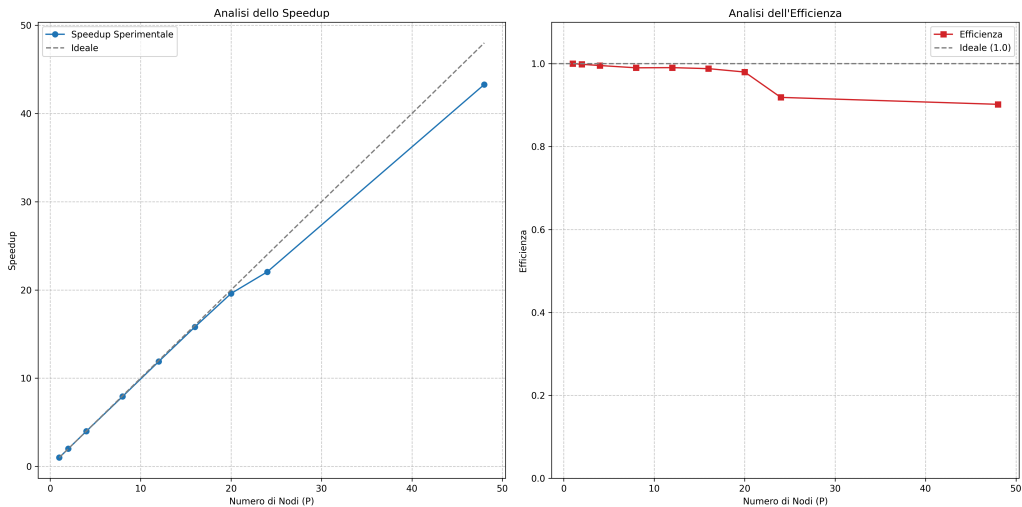


Figure 5: Strong Scaling - OpenMP

OpenMP (iii)



5.c.b Weak Scaling

- Il tempo di esecuzione rimane pressoché costante ($\approx 0.87s$), ogni core riceve la stessa **quantità** di lavoro.

P	N	T. Medio (s)	Std. dev	Speedup	Efficiency
1	5000	0.8681	0.0015	1	1
2	7071	0.8699	0.0004	1.9958	0.9979
4	10000	0.8683	0.0018	3.9988	0.9997
8	14142	0.8678	0.0023	8.0024	1.0003
12	17320	0.8731	0.002	11.9318	0.9943
16	20000	0.872	0.0021	15.928	0.9955
20	22360	0.8744	0.0028	19.8553	0.9928
24	24494	0.8793	0.002	23.6943	0.9873
48	34641	0.9444	0.0212	44.123	0.9192

OpenMP (iv)

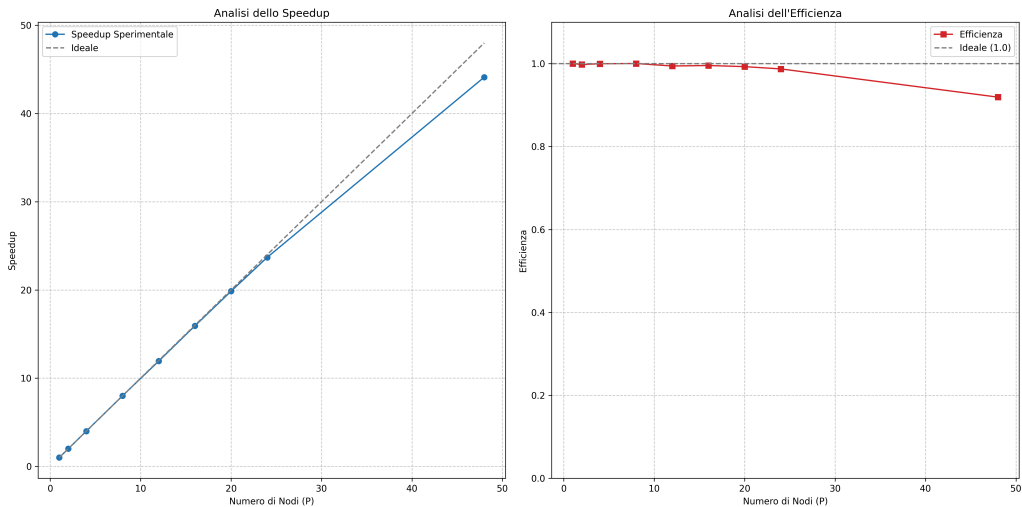


Figure 6: Weak Scaling - OpenMP

6 Conclusioni



6.a.a OpenMP (Shared Memory)

- **Speedup:** Quasi ideale (Efficienza sempre $> 95\%$).
- **Punti di forza:** Accesso diretto ai dati, zero overhead di comunicazione.
- **Criticità:** Vincolato alle risorse di un singolo nodo.

6.a.b MPI (Distributed Memory)

- **Speedup:** Limitato (Efficienza anche $< 20\%$).
- **Punti di forza:** Permette di eseguire su cluster multi-nodo per N estremamente grandi.
- **Criticità:** Performance limitate dall'overhead di comunicazione.

In base ai calcoli effettuati, **OpenMP** risulta la soluzione più efficiente per la natura del problema. **MPI** è necessario solo quando la memoria del singolo nodo non è più sufficiente. Soluzione possibile:

Hybridization